

语言基础第六天：

回顾：

1. break：跳出循环
continue：跳过循环体中剩余语句而进入下一次循环
2. 嵌套循环：
 - 循环中套循环，外层循环走一次，内层循环走所有次，嵌套层数越少越好，
 - break只能跳出当前一层循环
3. 数组：
 - 引用数据类型、相同数据类型元素的集合

```
int[] arr = new int[3]; //0,0,0
int[] arr = {2,3,7};
int[] arr = new int[]{2,3,7};
arr[0] = 100;
System.out.println(arr[arr.length-1]);
for(int i=0;i<arr.length;i++){
    arr[i] = (int)(Math.random()*100);
    System.out.println(arr[i]);
}
System.arraycopy(a,1,b,0,4);
int[] b = Arrays.copyOf(a,6);
a = Arrays.copyOf(a,a.length+1);
Arrays.sort(arr); //升序
```

精华笔记：

1. 方法：函数、过程
 - 作用：用于封装一段特定的业务逻辑功能
 - 建议：尽可能独立，一个方法只干一件事
 - 方法可以被反复多次调用
 - 好处：可以减少代码重复，有利于代码维护
 - 何时用：只要是一个独立的业务功能，就得封装到一个方法中
2. 方法的定义：五要素

```
修饰词 返回值类型 方法名(参数列表){
    方法体-----具体的业务逻辑功能实现
}
```

3. 方法的调用：
 - 无返回值：方法名(有参传参);
 - 有返回值：数据类型 变量 = 方法名(有参传参);
4. return：
 - return 值; //1)结束方法的执行 2)返回结果给调用方-----用在有返回值的方法中
 - return; //1)结束方法的执行-----用在无返回值的方法中

5. 方法的重载(overloading):

- 发生在同一类中，方法名相同，参数列表不同
- 编译器在编译时会根据方法的签名自动绑定方法

笔记:

1. 方法：函数、过程

- 作用：用于封装一段特定的业务逻辑功能
- 建议：尽可能独立，一个方法只干一件事
- 方法可以被反复多次调用
- 好处：可以减少代码重复，有利于代码维护
- 何时用：只要是一个独立的业务功能，就得封装到一个方法中

2. 方法的定义：五要素

```
修饰词  返回值类型  方法名(参数列表){  
    方法体-----具体的业务逻辑功能实现  
}
```

```
//无参无返回值  
public static void say(){  
    System.out.println("大家好，我叫WKJ，今年38岁了");  
}  
  
//有参无返回值  
public static void sayHi(String name){ //-----形参  
    System.out.println("大家好，我叫"+name+"，今年38岁了");  
}  
  
//有参有返回值  
public static void sayHello(String name,int age){ //-----形参  
    if(age>=60){ //在某种特定条件下，提前结束方法  
        return; //结束方法  
    }  
    System.out.println("大家好，我叫"+name+"，今年"+age+"岁了");  
}  
  
//有参有返回值  
public static int sum(int num1,int num2){  
    int num = num1+num2;  
    return num; //返回的是num里面的那个数  
    //return num1+num2; //返回的是num1与num2的和  
}  
  
//生成一个整型数组，并填充随机数据  
public static int[] generateArray(int len,int max){  
    Random rand = new Random();  
    int[] arr = new int[len];  
    for(int i=0;i<arr.length;i++){  
        arr[i] = rand.nextInt(max+1); //包括max  
    }  
    return arr;  
}
```

3. 方法的调用:

- 无返回值: 方法名(有参传参);

```
public static void main(String[] args) {
    say(); //调用say()方法

    //sayHi(); //编译错误, 有参则必须传参
    //sayHi(250); //编译错误, 参数类型必须匹配
    sayHi("zhangsan"); //String name="zhangsan" //-----实参
    sayHi("lisi"); //String name="lisi" //-----实参
    System.out.println("-----");

    sayHello("zhangsan",25); //-----实参
    sayHello("lisi",27); //-----实参
}
```

- 有返回值: 数据类型 变量 = 方法名(有参传参);

```
public static void main(String[] args) {
    int a = sum(5,6); //sum(5,6)的值就是return后的那个数
    System.out.println(a); //11---模拟对返回值的后续操作
    int m=5,n=6;
    int b = sum(m,n); //传的是m和n里面的数
    System.out.println(b); //11---模拟对返回值的后续操作
    System.out.println("-----");

    int[] c = generateArray(5,10); //模拟第1个人的访问
    System.out.println("数组的长度:"+c.length); //---模拟对返回值的后续操作
    for(int i=0;i<c.length;i++){ //---模拟对返回值的后续操作
        System.out.println(c[i]);
    }
    System.out.println("-----");

    int[] d = generateArray(10,30); //模拟第2个人的访问
    System.out.println("第1个元素的值:"+d[0]); //---模拟对返回值的后续操作
    for(int i=0;i<d.length;i++){ //---模拟对返回值的后续操作
        System.out.println(d[i]);
    }
}
```

4. return:

- return 值; //1)结束方法的执行 2)返回结果给调用方-----用在有返回值的方法中
- return; //1)结束方法的执行-----用在无返回值的方法中

5. 方法的重载(overloading):

- 发生在同一类中, 方法名相同, 参数列表不同
- 编译器在编译时会根据方法的签名自动绑定方法

```
package day06;
public class MethodDemo {
    public static void main(String[] args) {
        say(); //自动绑定无参say
        say("zhangsan"); //自动绑定String参的say
    }
}
```

```

        say("lisi",25); //自动绑定String+int参的say
        //say(3.456); //编译错误，没有double参的say
    }

    //无参无返回值
    public static void say(){
        System.out.println("大家好，我叫WKJ，今年39岁了");
    }

    //有参无返回值
    public static void say(String name){ //-----形参
        System.out.println("大家好，我叫"+name+"，今年39岁了");
    }

    //有参有返回值
    public static void say(String name,int age){ //-----形参
        if(age>=50){ //在某种特定条件下，提前结束方法
            return; //结束方法的执行
        }
        System.out.println("大家好，我叫"+name+"，今年"+age+"岁了");
    }

    public static void say(int age) {} //正确的重载
    public static void say(int age,String name){} //正确的重载
    //public static int say(){ return 1; } //编译错误，重载与返回值类型无关
    //public static void say(String address) { } //编译错误，重载与参数名称
    无关
}

```

6. 综合案例:

```

package day06;
import java.util.Scanner;
/**
 * 需求:<<主持人大赛>>有N位评委给选手打分
 * 选手的最终得分为:去掉最高分和最低分之后的N-2位评委的平均分
 * 要求:使用方法实现
 */
public class CalTotalAvg {
    public static void main(String[] args) { //撒贝宁
        double[] scores = inputData(6); //1)录入评委的评分
        double avg = calAvg(scores); //2)计算平均分
        System.out.println("平均分为:"+avg);
    }

    /** 该方法用于录入N位评委的评分 */
    public static double[] inputData(int n){
        double[] scores = new double[n]; //评分数组
        Scanner scan = new Scanner(System.in);
        for(int i=0;i<scores.length;i++){
            System.out.println("请录入第"+(i+1)+"位评委的分数:");
            scores[i] = scan.nextDouble();
        }
        return scores;
    }
}

```

```

    /** 该方法用于计算平均分 */
    public static double calAvg(double[] scores){
        double total = 0.0; //总分
        double max = scores[0]; //假设第1个元素为最高分
        double min = scores[0]; //假设第1个元素为最低分
        for(int i=0;i<scores.length;i++){
            total += scores[i]; //累加所有评分
            if(scores[i]>max){ //找最高分
                max = scores[i];
            }
            if(scores[i]<min){ //找最低分
                min = scores[i];
            }
        }
        //计算平均分---总分减掉最高分和最低分之后，再除以(评委数-2)
        double avg = (total-max-min)/(scores.length-2);
        return avg;
    }
}

package day06;
import java.util.Random;
import java.util.Scanner;
/**
 * 需求:猜数字小游戏
 * 训练目标: while(true)自造死循环+break
 */
public class Guessing {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        Random rand = new Random();
        int num = rand.nextInt(1000)+1; //1到1000
        System.out.println(num); //作弊

        while(true){ //自造死循环
            System.out.println("猜吧!");
            int guess = scan.nextInt();
            if(guess>num){
                System.out.println("猜大了");
            }else if(guess<num){
                System.out.println("猜小了");
            }else{
                System.out.println("恭喜你猜对了!");
                break; //跳出循环
            }
        }
    }
}

```

补充:

1. 形参: 形式参数, 定义方法时的参数为形参

实参：实际参数，调用方法时的参数为实参

2. 方法的签名：方法名+参数列表

3. Debug调试工具：

- 添加断点：点击你想添加断点那行的前面
- Debug运行程序，程序会自动进入第1个断点处，暂停
- F7：逐步调试(会进入到方法中)
 - F8：逐过程调试(不会进入到方法中)
 - F9：直接跳到下一个断点处，若后面没有断点了，则调试结束

4. 明日单词：

- 1) **Student**: 学生
- 2) **className**: 班级名称
- 3) **stuId**: 学号
- 4) **study**: 学习
- 5) **play**: 玩
- 6) **another**: 另一个
- 7) **car**: 小汽车
- 8) **brand**: 品牌
- 9) **color**: 颜色
- 10) **price**: 价格
- 11) **start**: 开始、启动
- 12) **run**: 跑
- 13) **stop**: 停止